

AI MANDI CHATBOT

Final Technical & Agentic Graph AI Contract

1. PROJECT OBJECTIVE

Build an AI-powered Mandi chatbot that allows users to ask natural-language questions about:

- Mandi arrivals
- Crop prices
- MSP
- Mandis
- Transportation/logistics
- Weather
- Relationships between these datasets

The chatbot must:

1. Understand natural language.
2. Handle common spelling mistakes.
3. Identify the user's intent.
4. Identify relevant datasets, columns, entities and dates.
5. Validate the request.
6. Perform calculations using the validated data.
7. Automatically decide whether a graph is useful.
8. Automatically select the correct graph type.
9. Render an interactive graph.
10. Generate a text summary alongside the graph.
11. Never invent numerical results.

The system is specifically designed to demonstrate the **Agentic Graph AI** capability required for the project bonus.

2. LLM

Primary LLM

Use:

Google Gemini 3.8 Flash

Model ID:

gemini-3.8-flash

Gemini 3.8 Flash is currently a stable model and supports:

- Structured outputs
- Function calling
- Thinking/reasoning
- Agentic workflows
- Long context
- Tool use

These capabilities make it suitable for the question-understanding, intent-generation and response-generation portions of this project.

Official model documentation:

□url□Gemini 3.8 Flash documentation□<https://ai.google.dev/gemini-api/docs/models/gemini-3.8-flash>□

3. LLM RESPONSIBILITY

Gemini is the **language and reasoning layer**.

Gemini is responsible for:

- Understanding natural-language questions
- Detecting intent
- Extracting entities
- Correcting common spelling mistakes
- Understanding dates
- Creating structured query JSON
- Requesting backend tools/functions
- Selecting/confirming visualization intent
- Generating the final natural-language explanation

Gemini is NOT the numerical source of truth.

4. DATA ANALYSIS ENGINE

Use:

Python + Pandas

The Pandas engine is responsible for:

- Filtering
- Grouping
- Aggregation
- Ranking
- Comparison
- Joining datasets
- Percentage calculations
- Trend calculations
- Correlation calculations
- Producing the actual chart data

Golden rule

Gemini = Understand

Backend = Validate

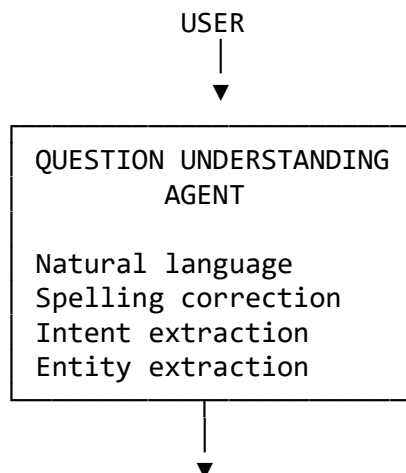
Pandas = Calculate

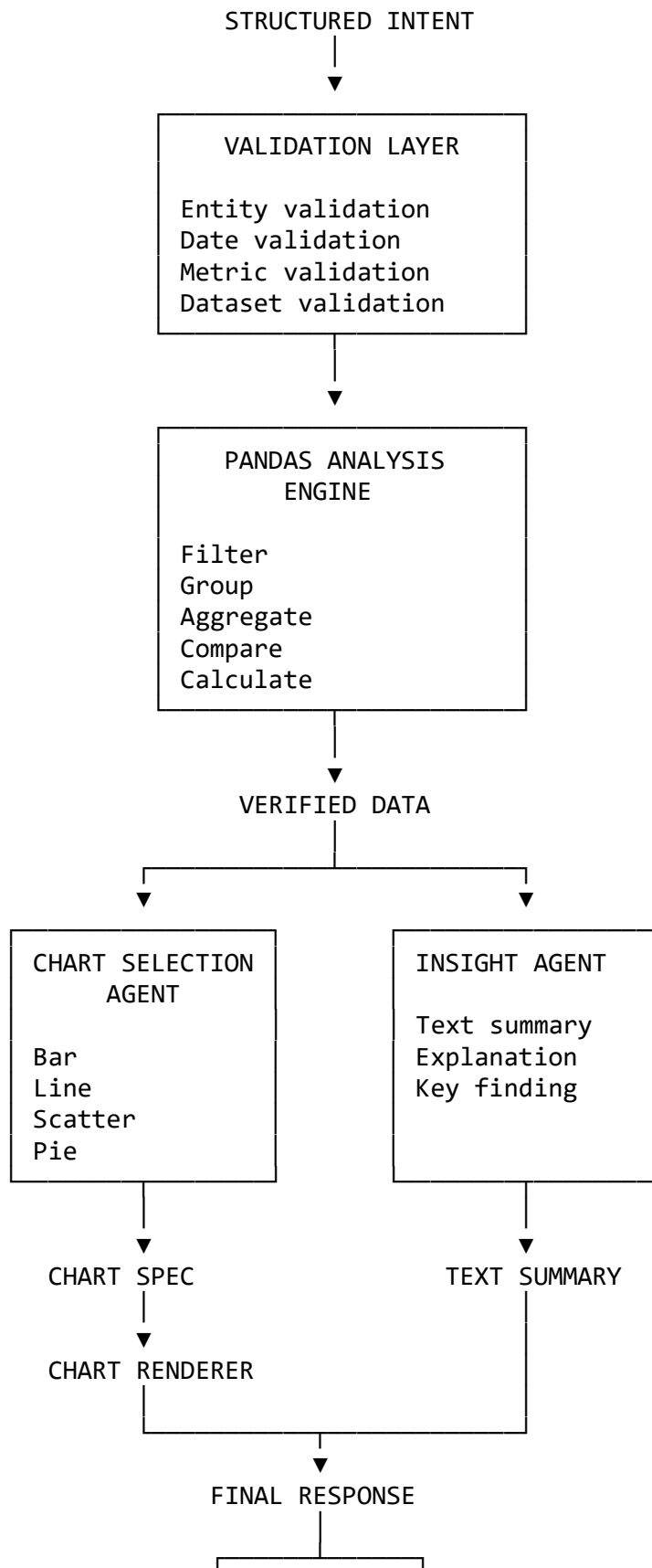
Chart Engine = Render

Gemini = Explain

Never ask the LLM to guess the answer when the answer can be calculated from the data.

5. COMPLETE ARCHITECTURE





6. SUPPLIED DATASETS

There are six files.

6.1 arrivals_final.csv

Purpose

Agricultural arrivals at mandis.

Actual dimensions

23,767 rows × 15 columns.

Columns

arrival_id
date
mandi_id
crop_name
variety
arrival_quantity
unit
farmer_count
quantity_numeric
arrival_quantity_qtl
quantity_quality
mandi_name
district
state
mandi_type

APPROVED analytical columns

arrival_id
date
mandi_id
crop_name
variety
farmer_count
arrival_quantity_qtl
mandi_name
district

state
mandi_type

Primary arrival metric

arrival_quantity_qtl

Use this for normal arrival calculations.

Do not normally use

arrival_quantity
unit
quantity_numeric
quantity_quality

These are raw/intermediate/quality fields.

7. price_and_msp_final.csv

Purpose

Market prices and MSP.

Actual dimensions

12,000 rows × 13 columns.

Columns

record_id
date
date_raw
mandi_id
district
district_final
crop_name
crop_name_raw
min_price
max_price
modal_price
msp
msp_was_imputed

APPROVED analytical columns

record_id
date
mandi_id
district_final
crop_name

min_price
max_price
modal_price
msp

Primary price metric

modal_price

If a user simply asks:

“What is the price?”

use modal_price.

If they specifically ask:

“What was the minimum price?”

use min_price.

If they ask:

“What was the maximum price?”

use max_price.

Quality/raw fields

date_raw
district
crop_name_raw
msp_was_imputed

Only use these when the user specifically asks about data quality or imputation.

8. transport_logistics_final.csv

Purpose

Transportation from mandis to warehouses.

Actual dimensions

10,000 rows × 22 columns.

Columns

trip_id
mandi_id
destination_warehouse
departure_time

arrival_time
transit_hours
distance
distance_unit
vehicle_no
driver_id
distance_km
departure_datetime
arrival_datetime
transit_hours_final
transit_hours_imputed
vehicle_no_raw
vehicle_no_clean
mandi_id_raw
mandi_name
district
state
mandi_type

APPROVED analytical columns

trip_id
mandi_id
destination_warehouse
distance_km
departure_datetime
arrival_datetime
transit_hours_final
vehicle_no_clean
mandi_name
district
state
mandi_type

Primary logistics metrics

distance_km
transit_hours_final

Raw/internal fields

Normally do not use:

departure_time
arrival_time
transit_hours
distance
distance_unit
vehicle_no
driver_id
transit_hours_imputed

vehicle_no_raw
mandi_id_raw

9. weather_daily.csv

Purpose

Daily weather summary.

Actual dimensions

252 rows × 5 columns.

Approved columns

date
avg_temperature_c
total_rainfall_mm
avg_humidity
sensor_count

This is the **primary weather dataset** for normal questions.

10. weather_sensors_final.csv

Purpose

Detailed sensor-level weather data.

Actual dimensions

15,000 rows × 17 columns.

Columns

sensor_id
timestamp
temperature
temp_unit
rainfall
rain_unit
humidity_percent
temperature_raw
temperature_numeric
temp_unit_clean
temperature_c
rainfall_raw

rain_unit_clean
rainfall_mm
timestamp_raw
timestamp_ist
date

APPROVED analytical columns

sensor_id
timestamp_ist
date
temperature_c
rainfall_mm
humidity_percent

Use this dataset when detailed sensor-level analysis is requested.

For normal daily weather questions, prefer `weather_daily`.

11. cross_dataset_recovery_audit.csv

Purpose

Data-cleaning/recovery audit.

Actual dimensions

5 rows × 7 columns.

dataset
operation
source
matching_key
missing_before
values_recovered
missing_after

This is NOT a normal analytical dataset.

Do not use it to answer ordinary price, arrival, weather or logistics questions.

It is documentation of the data-cleaning process.

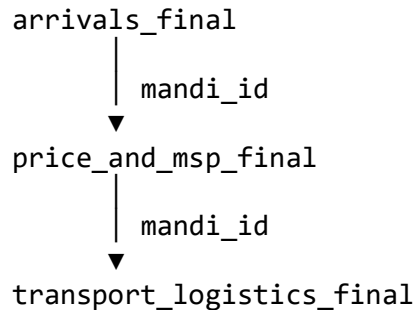
12. DATASET RELATIONSHIPS

Mandi relationship

The primary mandi key is:

mandi_id

Main relationship:



Crop relationship

Arrivals and prices share:

crop_name

The strongest analytical join between arrivals and prices should normally use:

date
mandi_id
crop_name

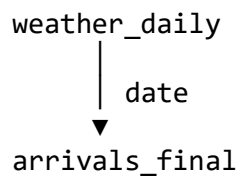
where the relevant values exist.

Date relationship

Weather can be connected to agricultural data using:

date

For example:



and:

```
weather_daily
  |
  | date
  v
price_and_msp_final
```

This enables cross-dataset analysis.

13. IMPORTANT DATA LIMITATION

Some records in the price dataset have missing `mandi_id`.

Therefore:

Never assign a missing mandi ID by guessing.

For example:

“What was the wheat price in Ludhiana?”

Only use records that can reliably be identified as Ludhiana.

14. APPROVED DIMENSIONS

The chatbot may filter/group by:

```
date
mandi_id
mandi_name
district
district_final
state
mandi_type
crop_name
variety
destination_warehouse
vehicle_no_clean
sensor_id
```

Only use a dimension when it exists in the relevant dataset.

15. APPROVED METRICS

Arrivals

arrival_quantity_qtl
farmer_count

Operations:

sum
average
minimum
maximum
count
ranking
percentage_change
trend

Prices

modal_price
min_price
max_price
msp

Operations:

average
minimum
maximum
ranking
difference
percentage_change
price_vs_msp
trend

Logistics

distance_km
transit_hours_final

Operations:

average
minimum
maximum
ranking
comparison

Weather

avg_temperature_c
total_rainfall_mm
avg_humidity

Operations:

average
minimum
maximum
sum
ranking
trend
correlation

16. SUPPORTED QUESTION TYPES

The chatbot must support:

1. Lookup

“What is the wheat price?”

2. Aggregation

“What was the average wheat price?”

3. Ranking

“Which mandi had the highest wheat arrivals?”

4. Top N

“Show the top 5 mandis by wheat arrivals.”

5. Bottom N

“Which 5 mandis had the lowest wheat prices?”

6. Comparison

“Compare wheat prices in Ludhiana and Khanna.”

7. Trend

“How did wheat prices change from January to September?”

8. Period comparison

“Compare wheat prices in January and August.”

9. MSP analysis

“Which crops are selling below MSP?”

10. Mandi analysis

“Tell me about Ludhiana mandi.”

11. Logistics

“Which mandi has the longest transit time?”

12. Warehouse analysis

“Which warehouse receives the most trips?”

13. Weather

“Which day had the highest rainfall?”

14. Cross-dataset analysis

“Did rainfall affect wheat arrivals?”

15. Relationship analysis

“Is temperature related to arrivals?”

16. Multi-condition analysis

“Which mandis had high arrivals but low prices?”

17. NATURAL-LANGUAGE UNDERSTANDING

The chatbot must understand different ways of asking the same thing.

For example:

What is the wheat price?

What's wheat selling for?

Show wheat rate.

Tell me the wheat mandi price.

All should resolve to approximately the same analytical intent.

The user should NOT have to know:

- dataset names
- column names
- Python
- SQL
- chart types

18. SPELLING / ENTITY RESOLUTION

The chatbot should support:

wheet → Wheat

ludhiyana → Ludhiana

Resolution order:

1. Exact match
2. Normalized match
3. Fuzzy match

If confidence is low:

ask the user for clarification.

Never make an uncertain guess.

19. DATE UNDERSTANDING

The chatbot should understand:

today

yesterday

last month

this month

January

January 2026

from March to June

first quarter

Convert natural-language dates into explicit date ranges before querying.

Available dataset period:

2026-01-01 → 2026-09-09

Do not claim data exists outside the available period.

20. INTENT JSON

Gemini should return structured JSON.

Example:

```
{
  "intent": "ranking",
  "datasets": ["arrivals"],
  "metric": "arrival_quantity_qtl",
  "aggregation": "sum",
  "filters": {
    "crop_name": "Wheat",
    "mandi_id": null,
    "mandi_name": null,
    "district": null,
    "state": null,
    "date_start": null,
    "date_end": null
  },
  "group_by": ["mandi_id"],
  "operation": "highest",
  "top_n": 5,
  "chart_required": true,
  "chart_type": "bar"
}
```

21. ALLOWED INTENTS

Only allow:

lookup
aggregation
ranking
comparison
trend
period_comparison
msp_analysis
logistics_analysis
weather_analysis
cross_dataset_analysis
mandi_summary
clarification
unsupported

22. ALLOWED DATASETS

arrivals
prices

transport
weather_daily
weather_sensors

Do not use the recovery audit as an ordinary analytical dataset.

23. ALLOWED AGGREGATIONS

sum
average
minimum
maximum
count
median

Only implement operations supported by the backend.

24. VALIDATION LAYER

Every structured intent must pass validation before reaching Pandas.

Validate:

Dataset

Does the requested dataset exist?

Metric

Is the requested metric approved?

Dimension

Is the requested grouping field valid?

Crop

Does the crop exist?

Mandi

Does the mandi exist?

Date

Is the date within the available range?

Operation

Is the requested calculation supported?

Join

If multiple datasets are requested, is there a valid relationship?

25. MISSING DATA RULES

Missing values are NOT zero.

NaN $\neq$ 0

Do not silently convert missing values to zero.

Do not invent missing values.

Do not assign missing mandi IDs.

26. IMPUTATION RULES

Relevant flags:

msp_was_imputed
transit_hours_imputed

For normal analysis, use the final standardized values.

If the user asks about data quality, explain when values were imputed.

27. AGENTIC GRAPH AI

This is a major project requirement because it targets the bonus criteria.

The chatbot should have three explicit agentic capabilities.

AGENT 1 — QUESTION UNDERSTANDING

Input:

User's natural-language question

Output:

Structured intent JSON

Responsibilities:

- Understand natural language
- Identify entities
- Correct spelling
- Identify intent
- Identify metrics
- Identify filters
- Identify date ranges
- Determine whether a graph is useful

Bonus criterion

Natural Language Understanding — 10 points

28. AGENT 2 — DATA ANALYSIS

The analysis engine receives the validated intent.

It:

Filters

↓

Joins

↓

Groups

↓

Aggregates

↓

Calculates

↓

Returns verified data

This agent/engine is deterministic.

It does not ask Gemini to calculate the actual answer.

29. AGENT 3 — CHART SELECTION

The chart-selection agent receives the verified result and chooses the correct chart.

Chart rules

BAR

Use for:

- Rankings
- Top N
- Bottom N
- Category comparisons
- Mandi comparisons
- Crop comparisons

Example:

“Which 5 mandis have the highest wheat arrivals?”

→ **Bar**

LINE

Use for:

- Time series
- Trends
- Changes over time
- Ordered dates

Example:

“Show wheat prices from January to September.”

→ **Line**

SCATTER

Use for:

- Relationships
- Correlation
- Two numerical variables

Example:

“Does rainfall affect wheat arrivals?”

→ **Scatter**

X:

total_rainfall_mm

Y:

arrival_quantity_qt1

PIE

Use only for simple part-to-whole composition with a small number of categories.

Example:

“What share of arrivals came from each crop?”

If there are too many categories, use a bar chart instead.

KPI

Use when the answer is primarily one number.

Example:

“What is the average wheat price?”

TABLE

Use for detailed records or when a table is more appropriate than a chart.

30. CHART SELECTION TABLE

Question type	Chart
Top 5 mandis	Bar
Highest/lowest mandi	Bar
Compare mandis	Bar
Compare crops	Bar
Price over time	Line
Arrivals over time	Line
Rainfall over time	Line

Question type	Chart
Temperature over time	Line
Rainfall vs arrivals	Scatter
Price vs arrivals	Scatter
Crop share	Pie
Single value	KPI
Detailed records	Table

The user normally should NOT have to specify the chart.

The system selects it automatically.

31. CHART RENDERING

After chart selection, generate a chart specification.

Example:

```
{
  "chart": {
    "type": "line",
    "title": "Wheat Modal Price Trend",
    "x": "date",
    "y": "modal_price",
    "data": []
  }
}
```

The frontend/chart renderer then renders the graph.

Charts should support:

- Interactive hover
- Tooltips
- Clear axis labels
- Titles
- Legends where required
- Responsive layout
- Appropriate number formatting

The chart must use the **actual calculated result**, never fabricated data.

32. TEXT SUMMARY AGENT

Every important chart-based response should include a text summary.

Example:

Wheat prices showed an overall upward trend during the selected period, increasing from ₹X to ₹Y.

Then:

[INTERACTIVE LINE CHART]

The summary must be generated from the actual calculated data.

33. FINAL RESPONSE FORMAT

For analytical questions:

📄 AI SUMMARY

[Short explanation of the result]

📄 VISUALIZATION

[Interactive chart]

Optional:

[Supporting table]

For simple questions that don't benefit from visualization:

📄 ANSWER

[Direct answer]

34. CROSS-DATASET ANALYSIS

Example:

“Did rainfall affect wheat arrivals?”

Process:

weather_daily
+

```
arrivals_final
  ↓
Filter crop = Wheat
  ↓
Group arrivals by date
  ↓
Join on date
  ↓
Calculate relationship
  ↓
Scatter chart
  ↓
Text summary
```

The system may report correlation.

It must NOT automatically claim causation.

Correct:

“Rainfall and wheat arrivals showed a negative correlation of X.”

Incorrect:

“Rainfall caused wheat arrivals to decrease.”

35. HIGH ARRIVALS / LOW PRICE

Question:

“Which mandis had high arrivals but low prices?”

The system must define the threshold.

Recommended:

High arrivals = $\geq$ 75th percentile

Low price = $\leq$ 25th percentile

The answer must state the threshold used.

36. FUNCTION CALLING

Gemini should interact with the backend through predefined functions/tools.

Conceptually:

Gemini
↓
query_data(...)
↓
Pandas
↓
verified result

Possible functions:

resolve_entity()
query_arrivals()
query_prices()
query_transport()
query_weather()
cross_dataset_analysis()
generate_chart_data()

The exact implementation is up to the developer.

37. SECURITY RULE

Do NOT allow Gemini to execute arbitrary Python generated from the user's question.

Preferred:

User
↓
Gemini
↓
Structured JSON
↓
Whitelist validation
↓
Predefined backend function
↓
Pandas

Not:

User
↓
Gemini
↓
arbitrary Python execution

38. RESPONSE OBJECT

Backend should return something similar to:

```
{
  "answer": "Khanna recorded the highest average wheat modal price.",
  "summary": "Khanna had the highest average wheat modal price among the selected mandis.",
  "data": [],
  "chart": {
    "type": "bar",
    "title": "Average Wheat Price by Mandi",
    "x": "mandi_name",
    "y": "modal_price",
    "data": []
  },
  "filters_applied": {
    "crop_name": "Wheat"
  },
  "data_range": {
    "start": "2026-01-01",
    "end": "2026-09-09"
  }
}
```

39. EXAMPLE END-TO-END QUERIES

Example 1 — Ranking

User:

Which 5 mandis had the highest wheat arrivals?

System:

Intent: ranking
Dataset: arrivals
Metric: arrival_quantity_qtl
Aggregation: sum
Group: mandi
Filter: Wheat
Chart: Bar

Output:

The top 5 mandis by wheat arrivals are:

[INTERACTIVE BAR CHART]

Example 2 — Trend

User:

Show wheat prices from January to September.

System:

Intent: trend
Dataset: prices
Metric: modal_price
Filter: Wheat
Group: date
Chart: Line

Output:

Wheat prices showed an overall trend of [actual finding].

[INTERACTIVE LINE CHART]

Example 3 — Correlation

User:

Does rainfall affect wheat arrivals?

System:

Datasets:
weather_daily + arrivals

Filter:
crop = Wheat

X:
total_rainfall_mm

Y:
arrival_quantity_qt1

Analysis:
correlation

Chart:
Scatter

Output:

Rainfall and wheat arrivals showed a [positive/negative] correlation of X during the available period. This indicates a relationship but does not establish causation.

[INTERACTIVE SCATTER CHART]

Example 4 — Spelling correction

User:

Which mandi had highest wheat price?

System:

wheet

↓

Wheat

Then execute the normal price-ranking workflow.

40. UNSUPPORTED QUESTIONS

If the data cannot answer a question:

Example:

“Predict wheat prices next year.”

Return:

“The current dataset supports historical analysis but does not contain a price-prediction model.”

Do not fabricate a prediction.

Similarly:

“What is today’s live mandi price?”

should not be presented as live information unless a live data source is explicitly connected.

41. BONUS POINT IMPLEMENTATION

The system should explicitly demonstrate the following three capabilities.

BONUS 1 — Natural Language Understanding

10 points

Demonstrate:

"Which mandi had the highest wheat arrivals?"
"What's wheat selling for?"
"Show wheat prices from January to September."
"Compare Ludhiana and Khanna."
"Did rainfall affect wheat arrivals?"

The system understands these without rigid commands.

BONUS 2 — Correct Chart Selection + Rendering

10 points

Demonstrate:

Ranking → Bar
Comparison → Bar
Trend → Line
Correlation → Scatter
Composition → Pie
Single value → KPI

The selected visualization must actually render using the calculated data.

BONUS 3 — Text Summary + Graph

10 points

For chart-worthy analytical questions:

🔍 AI SUMMARY

+

🔍 INTERACTIVE GRAPH

The summary must be based on the calculated result.

42. SUCCESS CRITERIA

The chatbot should be able to receive:

“Show me the top 5 mandis for wheat arrivals.”

and automatically:

1. Understand the question
2. Correct/resolve "wheat"
3. Identify ranking intent
4. Identify arrivals dataset
5. Identify arrival_quantity_qtl
6. Identify mandi as grouping
7. Query the validated data
8. Calculate top 5
9. Select Bar chart
10. Render interactive Bar chart
11. Generate text summary

The user should not need to specify:

- Dataset
- Column
- Aggregation
- Python
- SQL
- Chart type

43. FINAL DESIGN PRINCIPLE

The entire chatbot follows:

```text

UNDERSTAND

↓

VALIDATE

↓

CALCULATE

↓

VISUALIZE

↓

EXPLAIN

The LLM is the **language interface**.

Pandas is the **calculation engine**.

The validated datasets are the **source of truth**.

The chart engine is the **visualization layer**.

The final response combines:

**Answer + Interactive Graph + AI Summary**

This architecture is designed specifically to satisfy the Agentic Graph AI bonus requirements while keeping numerical answers grounded in the supplied validated data.